INFORME DE PRÁCTICA 2

Extracción de características

Visión por Ordenador I

AUTORES

Miguel Sobral Barrento
Alonso Bernabé Sanjuán

GRADO EN INGENIERÍA MATEMÁTICA E INTELIGENCIA ARTIFICIAL

Universidad Pontificia Comillas, ICAI

Curso 2026/27

1 · INICIO

Objetivo y preparación

En esta práctica hemos trabajado la detección de esquinas, la extracción de segmentos rectos y la detección y correspondencia de puntos de interés. El código se ejecuta sobre las cuatro imágenes del apartado A-B y sobre la imagen de objetos del apartado C.

2 · DESARROLLO

Apartado A · Detección de esquinas

Carga de imágenes y canales de color

Comparamos imageio.imread(), que entrega imágenes RGB, con cv2.imread(), que trabaja en BGR. Al guardar directamente el resultado de ImageIO con cv2.imwrite(), se intercambian los canales rojo y azul: en las imágenes del ZIP la pista azul aparece marrón, mientras que la lectura de OpenCV conserva el color azul. La celda que intenta guardar en ../data/partA devuelve False; el ZIP incluye las dos imágenes comparativas dentro de results/partA_processed.

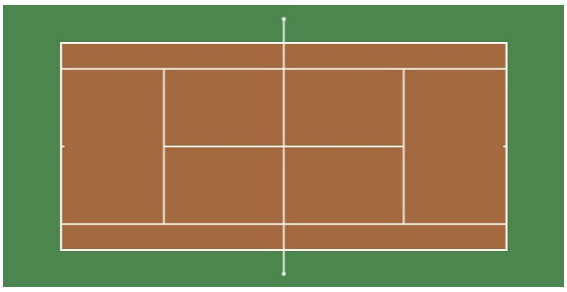
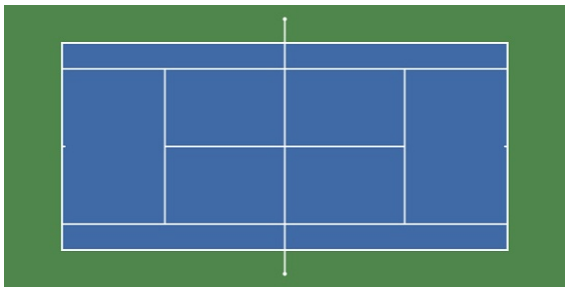
ImageIO (RGB) guardado con OpenCV	OpenCV (BGR)
	

Figura 1. Efecto del orden RGB/BGR al guardar la imagen de tenis.

Métodos de detección

Para Harris convertimos la imagen a escala de grises y float32, calculamos cv2.cornerHarris(), dilatamos la respuesta y marcamos en rojo los píxeles por encima del 1% de su máximo. Para Shi-Tomasi empleamos cv2.goodFeaturesToTrack() y dibujamos círculos de radio 4. En la teoría, Harris calcula la respuesta a partir de la traza y el determinante del tensor de estructura; Shi-Tomasi usa el menor de sus dos autovalores. Creamos results/partA_processed para guardar las salidas.

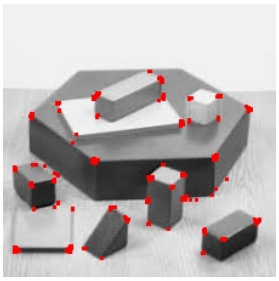
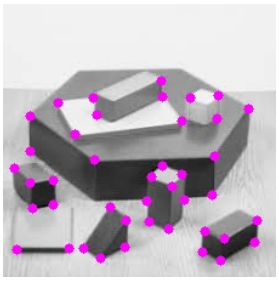
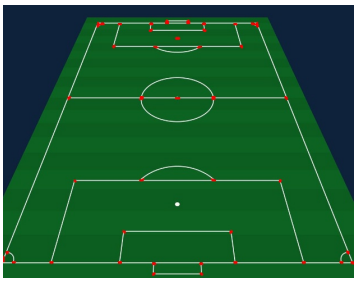
	Harris	Shi-Tomasi
Geometry		
Football		

Figura 2. Esquinas detectadas en geometry y football; los puntos de Harris son rojos y los de Shi-Tomasi aparecen en magenta o naranja.

2 · DESARROLLO

Apartado A · Preguntas A.1 y A.2

Pregunta A.1. Repetimos Harris sobre sudoku y tennis con blockSize=2, ksize=3 y k=0.04. En el sudoku se resaltan numerosos cruces de la cuadrícula; en la pista aparecen los encuentros y extremos de las líneas.

Pregunta A.2. Repetimos Shi-Tomasi con maxCorners=100, qualityLevel=0.1 y minDistance=10. Los círculos morados de sudoku y naranjas de tennis se concentran en intersecciones y cambios de dirección de las líneas.

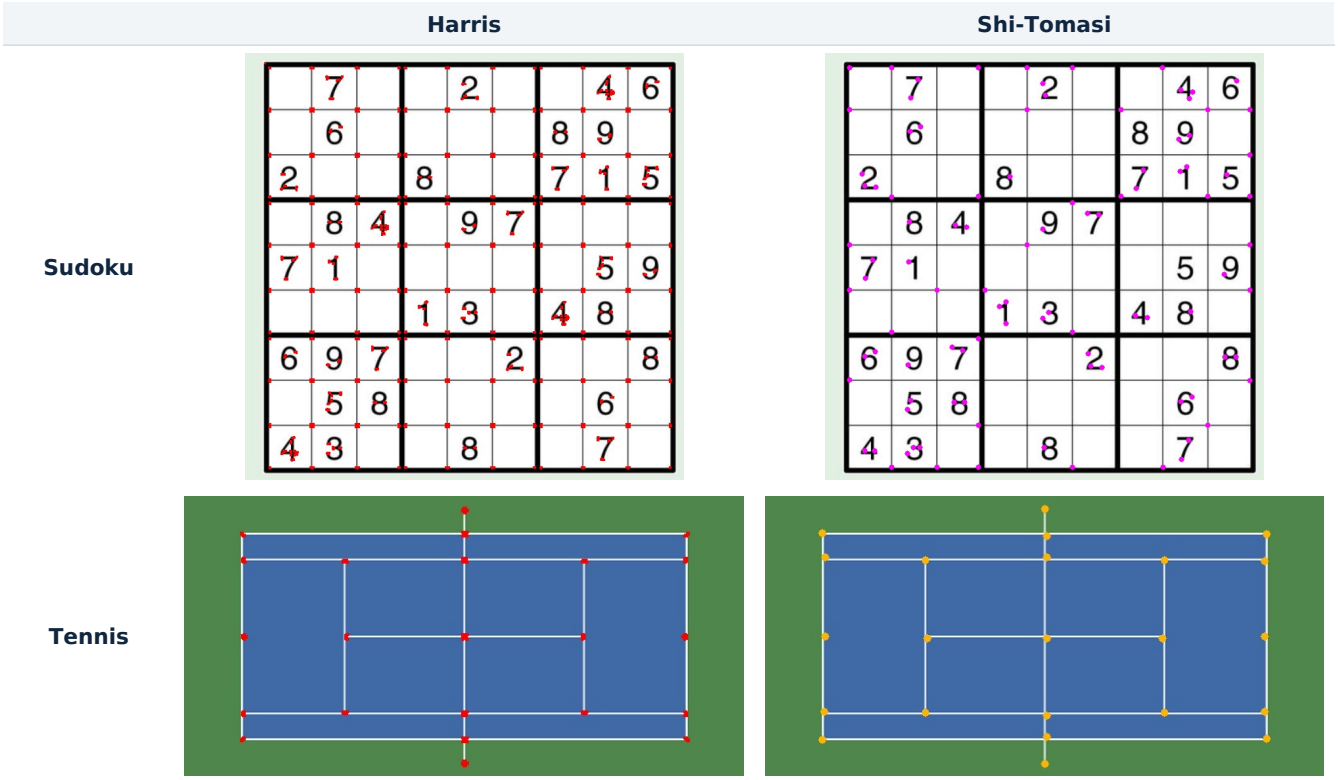


Figura 3. Resultados de las preguntas A.1 y A.2 en las dos imágenes restantes.

En las cuatro imágenes, los dos métodos señalan zonas con cambios de intensidad en más de una dirección, aunque la distribución y el número de puntos visibles dependen del criterio de respuesta y de sus parámetros. En las imágenes del sudoku y de las pistas, los cruces de líneas producen respuestas claras.

3 · DESARROLLO

Apartado B · Detección de líneas rectas

Convertimos cada imagen a escala de grises y aplicamos Canny con umbrales 50 y 150 para obtener mapas binarios de bordes. Después usamos cv2.HoughLinesP() sobre esos mapas y dibujamos los segmentos en verde sobre una copia de la imagen original. Se mantuvieron rho=1, theta=1 grado, threshold=20, minLineLength=10 y maxLineGap=5 en las cuatro imágenes, cuyos resultados se guardan en results/partB_processed.

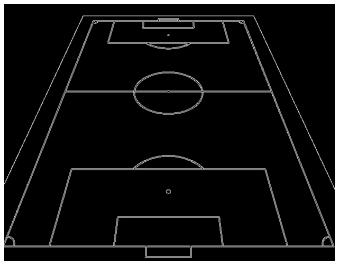
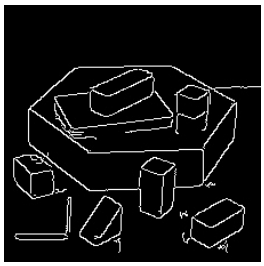
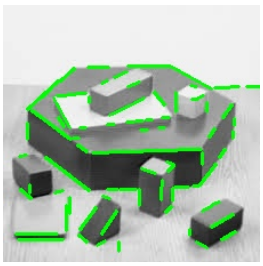
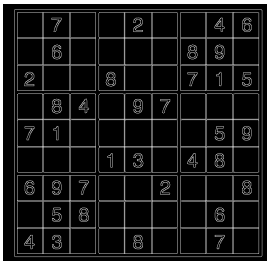
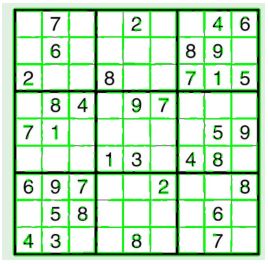
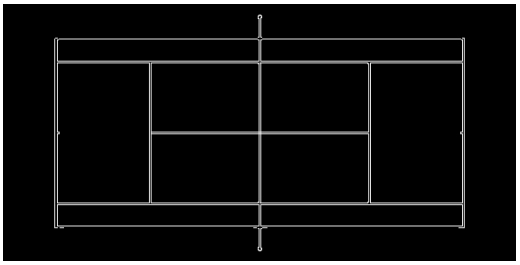
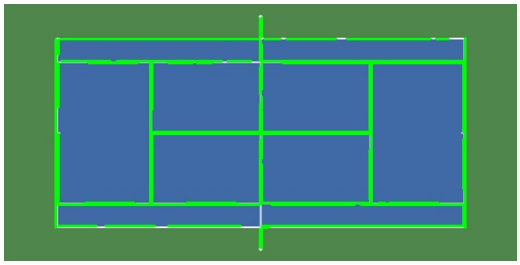
	Canny	Hough sobre la imagen original
Football		
Geometry		
Sudoku		
Tennis		

Figura 4. Bordes de Canny y segmentos detectados con Hough en las cuatro imágenes.

En football y tennis se recuperan las líneas principales del campo; en geometry aparecen segmentos de los contornos rectos de los objetos, y en sudoku predominan las divisiones de la cuadrícula. Canny aporta los píxeles de borde y Hough agrupa sus alineaciones en segmentos.

Pregunta B.1. El procedimiento se repitió en sudoku y tennis, como se muestra en las dos últimas filas de la Figura 4.

4 · DESARROLLO

Apartado C · Detección de puntos de interés

C.1 y C.2 · Espacio de escalas

Creamos results/partC/feature_extraction con subcarpetas para gaussianas, DoG, correspondencias y resultados SIFT. Cargamos 0.jpg en escala de grises, fijamos sigma inicial 1.6 e intervals=5 y generamos seis imágenes gaussianas aplicando sucesivamente cv2.GaussianBlur(). En la Figura 5 se observa el suavizado progresivo, que reduce los detalles finos al aumentar la escala.

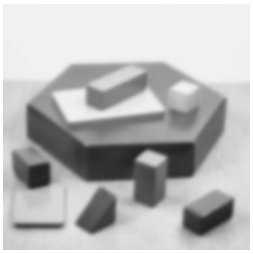
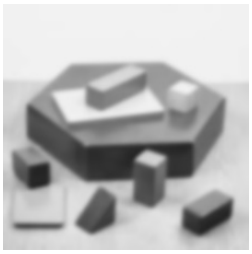
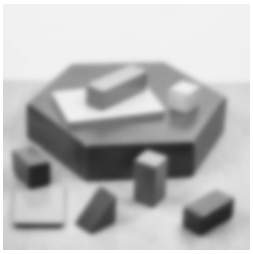
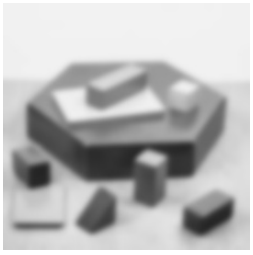
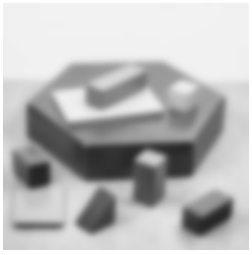
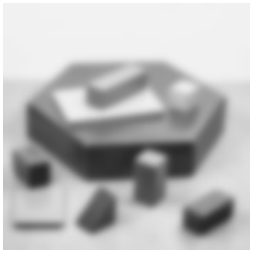
Gaussiana 0	Gaussiana 1	Gaussiana 2
		
		

Figura 5. Imágenes gaussianas del espacio de escalas (gaussian_0 a gaussian_5).

C.3 · Diferencias de gaussianas

Restamos cada par de gaussianas consecutivas con cv2.subtract(), obteniendo cinco imágenes DoG. Para visualizarlas como enteros sin signo se suma 128 y se recortan los valores al intervalo de imagen; por eso el fondo aparece gris y los cambios se aprecian como variaciones de intensidad.

DoG 0	DoG 1	DoG 2
		
		

Figura 6. Diferencias de gaussianas guardadas en el ZIP (la última celda se deja vacía).

C.4 · Evaluación de extremos

isPixelAnExtremum() compara el píxel central con sus 26 vecinos en tres regiones 3x3 consecutivas y exige que su valor absoluto supere 0.03. Así selecciona candidatos máximos o mínimos en el espacio de escalas.

4 · DESARROLLO

Apartado C · Localización y correspondencia

C.5 y C.6 · Puntos clave y descriptores

findScaleSpaceExtrema() recorre las DoG; los candidatos se refinan mediante un ajuste cuadrático, reciben una orientación y se eliminan duplicados. computeKeypointsAndDescriptors() integra estas etapas y genera los descriptores respecto a la orientación asignada, con el objetivo de tolerar giros.

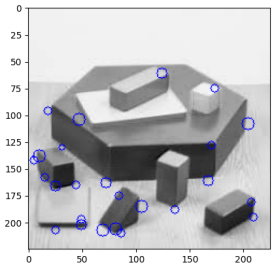
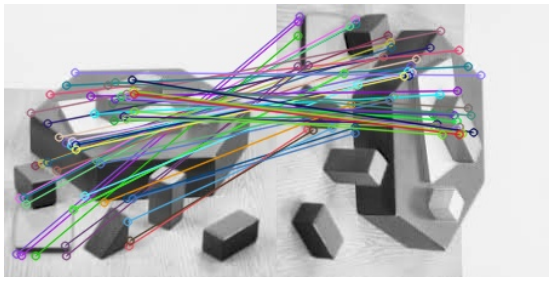
Puntos clave detectados	SIFT de OpenCV: 0.jpg y 0_rot.jpg
	

Figura 7. Detección de puntos clave y correspondencias en la imagen de prueba y su versión girada.

C.7 · Correspondencia en la imagen de prueba

El matcher propio usa FLANN, el criterio de Lowe (0.7) y una homografía estimada con RANSAC. En esta ejecución informa 0/10 correspondencias aceptadas y no genera figura; SIFT de OpenCV sí muestra matches entre 0.jpg y 0_rot.jpg.

Pregunta C.1 · Imágenes propias

Aplicamos ambos métodos a propia_1.jpg y propia_2.jpg, dos fotos de la misma página desde puntos de vista distintos. El pipeline propio dibuja sus coincidencias y el contorno transformado; OpenCV dibuja los 50 matches de menor distancia.

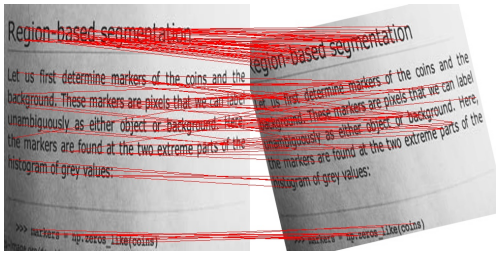
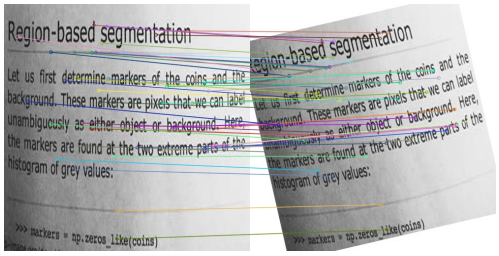
Pipeline propio	SIFT de OpenCV
	

Figura 8. Comparación de correspondencias en las fotografías propias.

Ambas salidas muestran correspondencias entre rasgos de la página. La comparación es cualitativa: el pipeline propio aplica FLANN y el criterio de Lowe, mientras que OpenCV ordena distancias y representa los 50 primeros matches. No hay correspondencias de referencia para medir la precisión.

5 · CIERRE

Conclusiones

Harris y Shi-Tomasi localizan esquinas; Canny y Hough permiten obtener segmentos rectos. El pipeline propio implementa las etapas principales de SIFT, pero no alcanza el mínimo de correspondencias en 0.jpg/0_rot.jpg; sí obtiene coincidencias en las imágenes propias. La comparación con OpenCV queda limitada a las salidas visuales.